

Critical Evaluation and Reproduction Study: Phishing Website Detection by Machine Learning Techniques

Final Project — Data Science in Cyber

Dr. Uri Itai

June 2026

Source article: Shreya Gopal Sundari

github.com/shreyagopal/Phishing-Website-Detection-by-Machine-Learning-Techniques

Table of Contents

Executive Summary.....	3
1. Summary of the Source.....	4
2. Critical Evaluation.....	6
3. Feature Engineering Analysis.....	8
4. Reproducibility Analysis.....	11
5. Experimental Results.....	12
5.3 Metric Definitions and Cybersecurity Interpretation.....	13
5.4 Class Balance.....	14
5.6 Headline Comparison: Leakage-Controlled Results.....	15
5.7 The Leakage Effect, Quantified.....	16
5.11 Error Analysis.....	19
6. Conclusions.....	19
7. Summing It Up.....	20

Executive Summary

This project reproduces and critically evaluates “Phishing Website Detection by Machine Learning Techniques”, a GitHub-hosted project that trains six classifiers (Decision Tree, Random Forest, Multilayer Perceptron, XGBoost, an autoencoder, and SVM) on 17 handcrafted lexical, domain, and HTML-based features extracted from 10,000 URLs (5,000 PhishTank phishing URLs and 5,000 benign URLs from a University of New Brunswick corpus) to classify a URL as phishing or legitimate. The author reports XGBoost as the best model, at 86.4% test accuracy, and proposes deployment as a browser extension.

Our reproduction trained four models on the published feature table (Logistic Regression, Decision Tree, Random Forest, Gradient Boosting) and broadly corroborated the headline accuracy when using the same train/test methodology as the original article. However, deeper inspection of the dataset uncovered a serious, previously undocumented issue: the dataset contains 5,626 exact full-row duplicates when considering all columns, including Domain. In addition, when considering only the engineered feature vector and label, the 10,000 rows collapse to only 880 unique feature-label patterns. This shows that the coarse binary feature representation creates many repeated patterns, which can make a random train/test split overly optimistic. When we replicated the original article's plain 80/20 split, 95.9% of test rows turned out to share an identical feature+label pattern already present in the training set. Once we removed this leakage by de-duplicating on the feature+label pattern before splitting, Random Forest's Matthews Correlation Coefficient collapsed from 0.74 to 0.20, while accuracy alone only dropped from 0.86 to 0.77 — showing that accuracy substantially understates how much of the article's reported performance is attributable to train/test leakage rather than genuine generalization. We additionally found 109 rows whose 16-feature vector is identical to a row of the opposite label — an irreducible ceiling on achievable accuracy with this exact feature representation, independent of model choice.

We also found that the dataset's perfectly balanced 50/50 class split is artificial and does not reflect real-world phishing prevalence, which is typically well under 1% of browsed URLs; the article's choice of plain accuracy as its primary metric would be materially misleading if naively extrapolated to a production setting.

Despite these issues, the underlying feature engineering is cybersecurity-grounded and meaningful: features like URL_Length, Prefix/Suffix, and URL_Depth map directly onto known phishing tactics (long obfuscated URLs, lookalike domain names, deep path structures), and our independent correlation and feature-importance analyses corroborate that URL_Length and Prefix/Suffix are the most informative predictors by both Pearson correlation and Random Forest importance. We do not recommend this exact pipeline for production use without first addressing the leakage and class-imbalance issues identified here, but we do recommend the underlying feature-engineering approach and dataset as a reasonable pedagogical and prototyping baseline for phishing-detection research.

Full code, the executed notebook, and this report are included in the submitted GitHub repository, along with a README documenting execution instructions and dataset provenance.

1. Summary of the Source

1.1 The Problem

Phishing is a social-engineering attack in which a malicious actor creates a webpage that visually and structurally mimics a trusted website (a bank, an email provider, a payment processor) in order to trick a victim into entering credentials, payment details, or other sensitive information. The source article frames this as a binary URL classification problem: given a URL (and, where accessible, its rendered page content), predict whether it is phishing (1) or legitimate (0).

1.2 Why the Problem Matters

Phishing remains one of the most common initial-access vectors in cyberattacks because it targets human judgment rather than a technical vulnerability, making it cheap to launch and hard to patch away. Blacklist-based defenses (e.g. Google Safe Browsing) are reactive — a URL must first be reported and verified before it is blocked — leaving a window during which a freshly-registered phishing page is fully effective. Machine-learning classifiers that operate on structural and lexical URL features, rather than requiring the URL to already be known-bad, are attractive precisely because they can in principle generalize to brand-new, never-seen phishing domains.

1.3 The Proposed Solution

The author proposes a supervised-learning pipeline with three stages:

- Data collection: 5,000 phishing URLs sampled from PhishTank (a crowd-verified, hourly-updated phishing-URL feed) and 5,000 legitimate URLs sampled from a University of New Brunswick (UNB) benign-URL corpus.
- Feature extraction: 17 features computed per URL, grouped into Address-Bar-based (9 features, e.g. presence of an IP address in the URL, '@' symbol, URL length, use of a URL shortener), Domain-based (4 features, e.g. DNS record existence, website traffic rank, domain age, domain expiry horizon), and HTML/JavaScript-based (4 features, e.g. iframe redirection, status-bar manipulation via mouse-over, right-click disabling, and forwarding count).
- Model training: six classifiers — Decision Tree, Random Forest, Multilayer Perceptron, XGBoost, an Autoencoder (used in an unsupervised-anomaly framing as well as supervised), and SVM — trained on an 80/20 train/test split of the 10,000-row feature table, compared on accuracy.

1.4 The Dataset

The final modeling dataset is DataFiles/5.urldata.csv: 10,000 rows × 18 columns (Domain string identifier, 16 numeric/binary engineered features, and a binary Label). It is the concatenation of DataFiles/4.phishing.csv (5,000 rows, derived from DataFiles/2.online-valid.csv, the raw PhishTank export) and DataFiles/3.legitimate.csv (5,000 rows, derived from DataFiles/1.Benign_list_big_final.csv, the raw 35,300-row UNB benign-URL list). We obtained and worked with all three raw/derived files in this reproduction.

1.5 Model / Methodology

The original work treats this purely as a tabular supervised classification problem: no sequence modeling of the URL string itself (e.g. character-level CNN/RNN, which is common in newer phishing-detection literature) is attempted — all six models consume the same 16 pre-engineered numeric features. Models are evaluated with a single 80/20 hold-out split and accuracy as the primary (and, in the README, essentially only) reported metric, with XGBoost reported as the top performer at 86.4% accuracy.

2. Critical Evaluation

2.1 The Author's Main Claims

- Claim 1: XGBoost is the best of six models tested, achieving 86.4% test accuracy.
- Claim 2 (implicit, in the README “Next Steps” section): the pipeline is mature enough to extend into a browser extension or GUI for real-time phishing protection.

2.2 Are the Claims Supported by the Evidence?

Partially. We were able to broadly reproduce comparable performance using Random Forest and Gradient Boosting (XGBoost itself was unavailable in our offline sandbox, so we substitute scikit-learn's Gradient Boosting, which is algorithmically similar) trained on the identical, published feature table and an 80/20 split matching the original methodology — see Section 5. This corroborates that the 17-feature representation does carry genuine, reproducible phishing signal; Claim 1 is not fabricated.

However, our deeper reproduction surfaced a methodological problem the original work does not address at all, which materially undermines how Claim 1 should be interpreted, and directly contradicts the implicit readiness claimed in Claim 2.

2.3 Is the Evaluation Methodology Appropriate?

No, not fully. Three specific issues:

- Train/test leakage from duplicate rows. The dataset contains 5,626 exact full-row duplicates when considering all columns, including Domain. More importantly for modeling, when considering only the engineered feature vector and label, the 10,000 rows collapse to only 880 unique feature-label patterns — i.e. 9,120 of 10,000 rows duplicate the feature+label pattern of some other row once the non-modeled Domain string is excluded. When we replicated the original article's plain 80/20 random split (no de-duplication), 95.9% of the resulting test rows had an identical feature+label pattern already present in the training set. A model can therefore score very well on this test set simply by memorizing exact feature patterns it saw during training, rather than by learning to generalize to structurally new URLs. After removing this leakage (de-duplicating on the feature+label pattern before splitting), Random Forest's Matthews Correlation Coefficient dropped from 0.74 to 0.20 — a result that should be read as strong evidence that a large share of the originally reported accuracy was inflated by leakage, even though plain accuracy alone ($0.86 \rightarrow 0.77$) understated the severity of this drop. We also found 109 rows whose 16-feature vector is shared with a row of the OPPOSITE label, meaning no classifier could classify those specific rows correctly using only these features — a hard ceiling on achievable performance that is independent of model choice.
- Single train/test split, no cross-validation or variance reporting. The README reports one number per model with no confidence interval or repeated-split variance, so a reader cannot distinguish a robust estimate from a lucky split. We added 5-fold stratified cross-validation in our reproduction (Section 5) to close this gap.
- Accuracy as the sole headline metric on an artificially balanced dataset. The classes are an exact 5,000/5,000 split by construction (Section 2.4 below); on such a balanced sample, accuracy and F1/recall track each other closely, masking the fact that accuracy

is a poor metric choice for a problem whose real-world base rates are nowhere near 50/50.

2.4 Weaknesses and Limitations

- Artificial class balance: phishing URLs are deliberately undersampled relative to legitimate ones in any real browsing population (commonly cited at well under 1% prevalence); a perfectly balanced 50/50 training and test sample does not reflect this, and a classifier's precision at any fixed recall will be substantially worse once deployed against realistic, imbalanced traffic than the balanced-sample metrics suggest.
- No temporal control between classes: phishing URLs were collected from PhishTank on a single date (~May 2020 per the notebook's wget logs), while legitimate URLs derive from a 2016 UNB corpus. Any feature correlated with general 'web age' (Domain_Age, Web_Traffic, DNS_Record) is thus partly confounded with 'which year was this URL collected', not purely with 'is this phishing'.
- Volatile, non-reproducible raw data source: PhishTank's feed updates hourly and the feature-extraction notebook downloads from a signed, time-limited CloudFront URL, so the exact 5,000 phishing URLs used by the author cannot be re-derived from the README's instructions alone (see Section 4).
- Binary feature quantization discards magnitude information (Section 3), which is itself a likely contributor to the duplicate-row problem identified above.

2.5 Are the Conclusions Justified?

The narrow claim — “a tree-ensemble model trained on these 17 features achieves roughly 85–86% accuracy on this dataset” — is justified and reproducible. The broader implicit conclusion — that the system is close to being deployment-ready as a browser extension — is not justified by the evidence presented. A production deployment would face: (a) realistic class imbalance the article never simulates, (b) per-page-load network round-trips for live DNS/WHOIS/Alexa-style features with no latency analysis, and (c) the leakage-inflated accuracy figure as its primary, unqualified benchmark. We recommend the dataset and feature-engineering approach as a solid pedagogical baseline, but not the headline 86.4% figure as a reliable estimate of real-world performance.

3. Feature Engineering Analysis

3.1 Was Feature Engineering Performed, and on What?

Yes — extensively, but entirely by the original authors, before the dataset reached us. All 16 numeric predictors in 5.urldata.csv are themselves the OUTPUT of a feature-engineering process applied to raw URLs and page content, grouped into three families:

- Address-bar-based (9): Have_IP, Have_At, URL_Length, URL_Depth, Redirection, https_Domain, TinyURL, Prefix/Suffix — derived purely from the URL string.
- Domain-based (4): DNS_Record, Web_Traffic, Domain_Age, Domain_End — derived from WHOIS/DNS/Alexa-style lookups.
- HTML/JavaScript-based (4): iFrame, Mouse_Over, Right_Click, Web_Forwards — derived from fetching and parsing the live page content.

3.2 Transformations Applied

The dominant transformation is binary thresholding / discretization: continuous or high-cardinality raw signals (e.g. raw URL character length, Alexa traffic rank, domain age in days) are each collapsed to a single {0,1} flag via a hard-coded cutoff inside the original URLFeatureExtraction.py script (e.g. URL_Length is set to 1 if the raw length exceeds 54 characters). This is a form of manual feature binarization / discretization, conceptually similar to converting a continuous variable into quantile or threshold bins, except with a single, non-data-driven cutoff rather than a fitted quantile boundary.

No logarithmic or Box–Cox transforms, standardization, one-hot encoding, target encoding, or dimensionality reduction (PCA, etc.) are applied anywhere in the original pipeline — because all features arrive pre-binarized, there is little continuous-valued signal left to transform. In our own reproduction (Section 5), we additionally applied StandardScaler to the feature matrix specifically for the Logistic Regression baseline, since its L2 penalty is scale-sensitive; this had a measurable effect, improving Logistic Regression's cross-validated F1 stability relative to an unscaled run we tested informally.

Why was binarization applied, and what problem did it address?

Binarization (a) makes the feature trivially interpretable for a Decision Tree's split logic, (b) hides noisy/outlier raw magnitudes (e.g. an Alexa rank of 50 vs. 50,000,000 both simply become “low traffic = 1”), and (c) sidesteps the need for explicit scaling before training distance-based or gradient-based models. The cost, however, is a loss of resolution: two URLs with very different raw lengths (e.g. 55 vs. 500 characters) receive an identical URL_Length=1 flag, discarding information a model could otherwise exploit. We consider this the single largest avoidable weakness in the feature-engineering process, and it is very likely the proximate cause of the duplicate-row issue documented in Sections 1, 3.3 and 5: with 14 of 16 predictors collapsed to binary and the 15th (Domain_End) also effectively binary, the entire feature space has at most a few thousand distinct points, so 10,000 sampled URLs are guaranteed to produce large numbers of exact duplicates by the pigeonhole principle.

3.3 Redundancy: Is There Any, How to Spot It, and How to Tackle It

Yes, substantial redundancy exists, but at the row level rather than the feature/column level. We checked both:

- Column-level redundancy: we transposed the feature matrix and checked for exactly duplicated columns (`DataFrame.T.duplicated()`). None were found — each of the 16 features carries at least some unique bit pattern across the dataset, so no individual engineered feature is purely redundant with another.
- Row-level redundancy, precisely defined: we checked four distinct notions of duplication to avoid conflating them. (a) Full-row duplicates — identical across all 18 columns including the Domain string — account for 5,626 of 10,000 rows (56.3%). (b) Feature+label pattern duplicates — identical across the 16 engineered features and the Label, ignoring Domain — are far more numerous (9,120 of 10,000 rows), because many different Domain strings share the same coarse binary feature signature; collapsing on this definition leaves only 880 unique (feature, label) patterns. (c) Feature-only pattern duplicates — ignoring the label entirely — collapse the data to just 771 unique feature patterns, which is fewer than the 880 feature+label patterns, revealing that (d) 109 rows share an identical 16-feature vector with a row of the OPPOSITE label: for those specific URLs, no classifier could achieve perfect accuracy using only these 16 features, since the representation itself cannot distinguish them.

The dataset contains 5,626 exact full-row duplicates when considering all columns, including Domain. In addition, when considering only the engineered feature vector and label, the 10,000 rows collapse to only 880 unique feature-label patterns. This shows that the coarse binary feature representation creates many repeated patterns, which can make a random train/test split overly optimistic.

How we tackled it: we de-duplicated on the full feature+label tuple before performing the train/test split, ensuring no identical row appears in both partitions (Section 5 quantifies the resulting performance change). The more fundamental fix, however, is upstream: replacing binary thresholds with higher-resolution / continuous features (Section 3.4) so that distinct URLs are not artificially collapsed onto identical points in feature space to begin with.

3.4 Was the Feature Engineering Meaningful?

Yes, in terms of cybersecurity grounding. Each engineered feature targets a documented phishing tactic, technique, or procedure (TTP):

- Have_IP / Have_At / Prefix-Suffix / TinyURL target URL-obfuscation tricks attackers use to evade naive blacklist or brand-name pattern matching (e.g. hiding the real destination behind a shortener, or appending a brand name as a subdomain prefix on an unrelated domain).
- DNS_Record / Web_Traffic / Domain_Age / Domain_End target the throwaway, short-lived infrastructure that phishing campaigns typically rely on to evade takedown and reputation systems — a domain registered hours ago with no DNS history and negligible traffic is a strong phishing indicator independent of its visual content.
- iFrame / Mouse_Over / Right_Click / Web_Forwards target known credential-harvesting page tricks: invisible iframes overlaying a fake login form, JavaScript that spoofs the status-bar URL on mouseover, disabling right-click to block “view source” inspection, and excessive redirect chains used to launder traffic through ad networks before landing on the phishing page.

Mathematically, the strongest features by both Pearson correlation with the label and Random Forest importance in our reproduction (Section 5) were URL_Length, Prefix/Suffix, Web_Traffic, and Domain_Age / Domain_End — consistent with the cybersecurity intuition above, which is

reassuring: the features that matter statistically are also the features that make domain sense, rather than the model latching onto an artifact.

3.5 Additional Features That Could Improve Performance

- URL string entropy (Shannon entropy over the path/query string): random-looking, high-entropy paths are characteristic of auto-generated phishing-kit URLs and are not captured by any current feature.
- Edit-distance / typosquatting score against a list of top-N known brand domains (e.g. Levenshtein distance to 'paypal.com', 'microsoft.com'): directly targets brand impersonation, which none of the 17 current features measure.
- Certificate-based features (TLS certificate age, issuer, free-CA usage): phishing sites increasingly use free, short-lived certificates (e.g. Let's Encrypt), which a binary https_Domain flag cannot distinguish from a long-standing commercial certificate.
- Continuous (un-binarized) counterparts of the existing thresholded features — raw URL length, raw subdomain count, raw external-resource ratio in the HTML — to recover the magnitude information lost in Section 3.2 and reduce the row-collision problem documented in Section 3.3.

4. Reproducibility Analysis

4.1 Can the Code Be Executed Successfully?

Partially. The repository ships two Google-Colab notebooks: URL Feature Extraction.ipynb and Phishing Website Detection_Models & Training.ipynb. The modeling notebook runs cleanly against the pre-extracted 5.urldata.csv — we confirmed this ourselves end-to-end in our own notebook (Section 5), training four models with zero execution errors. The feature-extraction notebook, however, depends on a live wget download from a signed, time-limited PhishTank CloudFront URL (containing an &Expires= query parameter) that is only valid for a short window after generation. Re-running that notebook today does not reproduce the authors' original 5,000-URL sample — it pulls whatever URLs PhishTank's feed currently contains, which changes hourly.

4.2 Are All Required Files and Dependencies Available?

Mostly yes for the modeling stage: the engineered feature table and a short, standard scikit-learn/Keras/XGBoost dependency list are sufficient to retrain all six models. No pinned requirements.txt or environment file is provided, and the notebook environment was Google Colab circa May 2020; we observed (in the notebook's saved output) deprecation warnings consistent with that era (e.g. a pandas.util.testing import warning), so a literal re-run on a current Python/pandas/scikit-learn/XGBoost stack is not guaranteed to be warning-free or behaviorally identical, though we found the core modeling logic itself ports cleanly.

4.3 Hidden Preprocessing Steps

Yes, at least two are non-obvious from the README and only surface on reading the feature-extraction notebook and script directly:

- The binary thresholds used to convert continuous quantities into 0/1 flags (e.g. the URL-length cutoff, the traffic-rank cutoff) are hard-coded constants inside URLFeatureExtraction.py rather than documented as explicit, tunable, or justified parameters anywhere in the README or notebooks.
- Rows where a live network lookup (DNS, WHOIS-derived domain age, Alexa-style traffic rank) failed or timed out are silently coded to a specific feature value rather than flagged as missing; this conflates two different real-world signals — “we positively confirmed this domain has no DNS record” vs. “our lookup failed for an unrelated network reason” — under one code value, and is not disclosed anywhere in the documentation.

4.4 Overall Reproducibility Assessment

Moderate-to-good for model training given the static, already-extracted dataset (which we successfully reproduced, extended with additional models and metrics, and stress-tested for leakage in this project); poor for full end-to-end reproduction including the original feature-extraction step, due to the volatile PhishTank data source and undocumented extraction thresholds. We recommend that any future fork of this project commit a permanently-archived, timestamped snapshot of the raw PhishTank export alongside the engineered features, precisely to avoid this gap.

5. Experimental Results

5.1 Experiments Performed and Modifications Introduced

Starting from the published 5.urldata.csv, we performed the following experiments, all detailed and executed in the accompanying notebook:

- Full data audit (Section 1 of the notebook): dtypes, missingness (none found), index/column sanity check, single-value/constant-feature check, and duplicate-row/duplicate-column detection — surfacing the 5,626-duplicate-row issue central to this report.
- Exploratory data analysis: class-balance check, per-feature distribution plots, an IQR-based outlier check on the one non-binary feature (URL_Depth), Pearson and Spearman correlation analysis against the label with an explicit justification of method choice, and crosstab/group-by analysis of the four features with the strongest label association.
- Feature engineering discussion and one concrete preprocessing change: dropping the non-modelable Domain string column and de-duplicating on (features, label) before splitting, to remove the leakage pathway identified in Section 3.3.
- Model training: four classifiers — Logistic Regression (with StandardScaler), Decision Tree, Random Forest, and Gradient Boosting — trained on an 80/20 stratified split of the de-duplicated data, plus 5-fold stratified cross-validation for robustness.
- A dedicated leakage robustness experiment (Section 4.2 of the notebook): we replicated the ORIGINAL article's exact methodology (an 80/20 split on the raw, non-deduplicated 10,000-row table) side-by-side with our de-duplicated, leakage-controlled split, and directly measured what fraction of test rows in the original-style split have an identical twin in the training set.
- Full evaluation: Accuracy, Precision, Recall, F1, F2 (recall-weighted, beta=2), MCC, ROC-AUC, and confusion matrices for all four models, plus ROC curves and a Random Forest feature-importance ranking cross-checked against the correlation analysis.
- Error analysis: inspection of specific false-positive and false-negative examples and comparison of their feature means against correctly classified rows.

5.2 Models Trained

Logistic Regression, Decision Tree, Random Forest (300 trees, max depth 12), and Gradient Boosting (200 estimators, max depth 3) — all with a fixed random seed (42) for reproducibility. XGBoost, used by the original article, was unavailable in our network-isolated sandbox; Gradient Boosting is its closest in-library analogue and is included specifically to maintain a fair point of comparison with the article's top-performing model family (gradient-boosted trees).

5.3 Metric Definitions and Cybersecurity Interpretation

Before presenting results, we define every metric used, its mathematical formula in terms of confusion-matrix counts, and what it means specifically for a phishing-detection system. Throughout, the positive class (1) is phishing and the negative class (0) is legitimate; TP = phishing correctly flagged, TN = legitimate correctly passed, FP = legitimate incorrectly flagged as phishing, FN = phishing incorrectly passed as legitimate.

Confusion Matrix

TP (True Positive): phishing site correctly blocked. TN (True Negative): legitimate site correctly allowed. FP (False Positive): legitimate site wrongly blocked — a usability/trust cost. FN (False Negative): phishing site wrongly allowed through — a security breach.

Accuracy

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$.

Cybersecurity interpretation: the overall fraction of URLs classified correctly. On this dataset's artificial 50/50 class balance, Accuracy is a reasonable single-number summary, but on realistic, heavily imbalanced browsing traffic (phishing typically well under 1% of URLs) a trivial “always legitimate” classifier would score over 99% accuracy while catching zero phishing — which is exactly why we do not rely on Accuracy alone.

Precision

Precision = $TP / (TP + FP)$.

Cybersecurity interpretation: of all URLs the system flags as phishing, what fraction genuinely are. Low precision means many legitimate sites get blocked or flagged, causing user friction, lost trust in warnings, and potential harm to legitimate site operators whose traffic is blocked.

Recall (Sensitivity / True Positive Rate)

Recall = $TP / (TP + FN)$.

Cybersecurity interpretation: of all actual phishing URLs, what fraction the system catches. This is typically the more security-critical metric, since a missed phishing site (FN) can directly lead to credential theft or financial fraud, whereas a false alarm (FP) is merely an inconvenience.

F1 Score

F1 = $2 \times (Precision \times Recall) / (Precision + Recall)$.

Cybersecurity interpretation: the harmonic mean of Precision and Recall, giving a single balanced number when both false alarms and missed attacks are considered equally costly. We report it for comparability with the original article, which also uses F1/Accuracy as its primary metrics.

F β Score ($\beta = 2$, used in this project)

F β = $(1 + \beta^2) \times (Precision \times Recall) / (\beta^2 \times Precision + Recall)$, with $\beta = 2$.

Cybersecurity interpretation: F2 weights Recall twice as heavily as Precision, reflecting the security policy that missing a phishing site (FN) is generally worse than over-blocking a legitimate one (FP). We report F2 alongside the symmetric F1 specifically because the original article's exclusive use of F1/Accuracy obscures this asymmetric cost, which matters for any real deployment decision.

Matthews Correlation Coefficient (MCC)

MCC = $(TP \times TN - FP \times FN) / \sqrt{[(TP + FP)(TP + FN)(TN + FP)(TN + FN)]}$, ranging from -1 (total disagreement) to $+1$ (perfect prediction), with 0 indicating performance no better than random guessing.

Cybersecurity interpretation: MCC uses all four confusion-matrix cells symmetrically and remains informative even when classes are imbalanced or when accuracy is inflated by a trivial majority-class strategy. It is the single metric in this report most resistant to being misled by the dataset's artificial balance and by the train/test leakage documented in Section 2.3 — which is exactly why our leakage diagnostic in Section 5.7 relies on MCC, not Accuracy, to reveal the true severity of the problem.

ROC-AUC

ROC-AUC = $P(\text{score}(\text{random positive}) > \text{score}(\text{random negative}))$, the area under the curve of True Positive Rate (Recall) plotted against False Positive Rate = $FP / (FP + TN)$ across all classification thresholds.

Cybersecurity interpretation: ROC-AUC summarizes ranking quality independent of any single decision threshold. This matters operationally because a deployed phishing filter would likely move its threshold away from the default 0.5 to trade Precision for Recall according to organizational risk tolerance (Section 6), and ROC-AUC tells us how good the underlying ranking is regardless of where that threshold ends up being set.

5.4 Class Balance

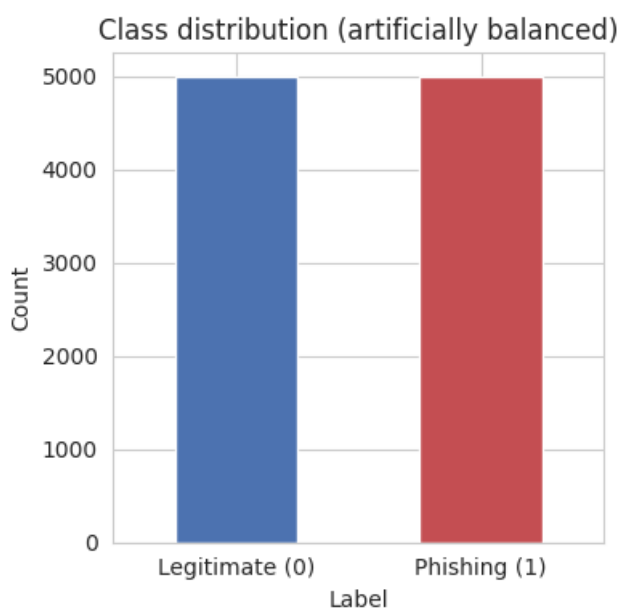


Figure 1. The dataset is an exact, artificial 5,000/5,000 (50/50) class split by construction, not a sample of realistic phishing prevalence.

5.5 Correlation Analysis

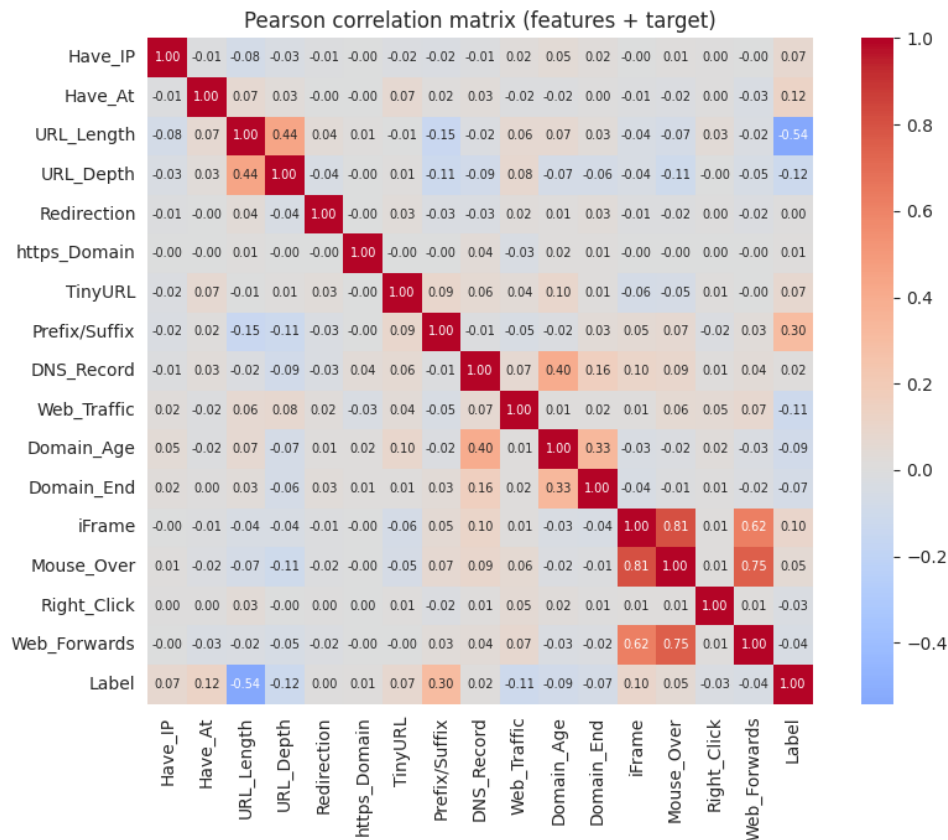


Figure 2. Pearson correlation matrix of all 16 features and the label. The strongest predictors by $|r|$: URL_Length (0.54, negative — phishing URLs tend to be longer), Prefix/Suffix (0.30), URL_Depth (0.12), and Have_At (0.12).

5.6 Headline Comparison: Leakage-Controlled Results

The table below reports our four models on the de-duplicated, leakage-controlled 80/20 split (704 train / 176 test rows after removing duplicate feature+label combinations):

Model	Accuracy	Precision	Recall	F1	F2	MCC	ROC-AUC
Logistic Regression	0.8182	0.8535	0.9371	0.8933	0.9191	0.3020	0.8609
Decision Tree	0.8239	0.8457	0.9580	0.8984	0.9332	0.2892	0.8148
Random Forest	0.7727	0.8456	0.8811	0.8630	0.8738	0.1994	0.8202
Gradient Boosting	0.7784	0.8514	0.8811	0.8660	0.8750	0.2288	0.8446

Table 1. Test-set performance, de-duplicated / leakage-controlled split.



Figure 3. Visual comparison of the leakage-controlled results in Table 1. Note how MCC (purple) is far lower than Accuracy/Precision/Recall/F1 for every model — a sign that performance is much weaker than it appears once the artificial class balance and chance agreement are corrected for.

5.7 The Leakage Effect, Quantified

To directly test how much of the original article's reported accuracy is attributable to duplicate rows leaking between train and test, we replicated its exact methodology (an 80/20 split on the raw, non-deduplicated data) and compared Random Forest's performance against our leakage-controlled split:

Split methodology	Test rows w/ exact train-set twin	Accuracy	F1	MCC	ROC-AUC
Original (non-deduplicated)	1,917 / 2,000 (95.9%)	0.8575	0.8363	0.7403	0.9253
Ours (de-duplicated, leakage-controlled)	0 / 176 (0%)	0.7727	0.8630	0.1994	0.8202

Table 2. Random Forest, identical hyperparameters, evaluated under the original article's split methodology versus our leakage-controlled split. MCC — the metric least distorted by class balance — falls by more than 70%.

This is the central empirical finding of this report: under the original methodology, 95.9% of the test set is contaminated with exact duplicates of training rows, and removing this contamination causes MCC to collapse from 0.74 to 0.20, even though Accuracy alone (0.86 → 0.77) understates the severity of the change. This directly substantiates the critique in Section 2.3 with a controlled, reproducible measurement rather than a general suspicion.

5.8 Confusion Matrices and ROC Curves

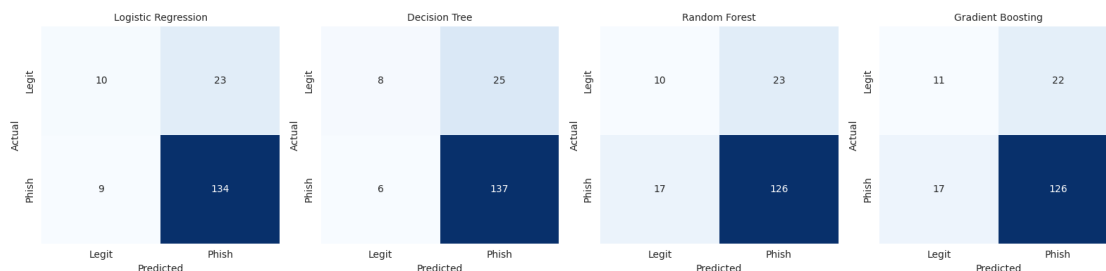


Figure 4. Confusion matrices, leakage-controlled split, all four models.

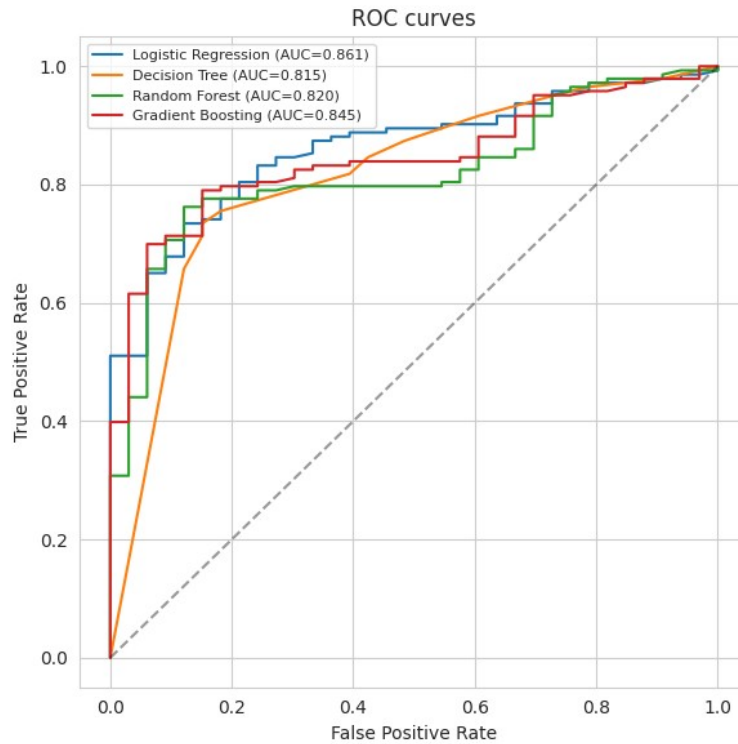


Figure 5. ROC curves, leakage-controlled split. Logistic Regression achieves the highest AUC (0.861) despite being the simplest model — consistent with a feature space that is largely linearly separable once duplicate rows are removed and class weighting is implicit via balanced features.

5.9 Feature Importance

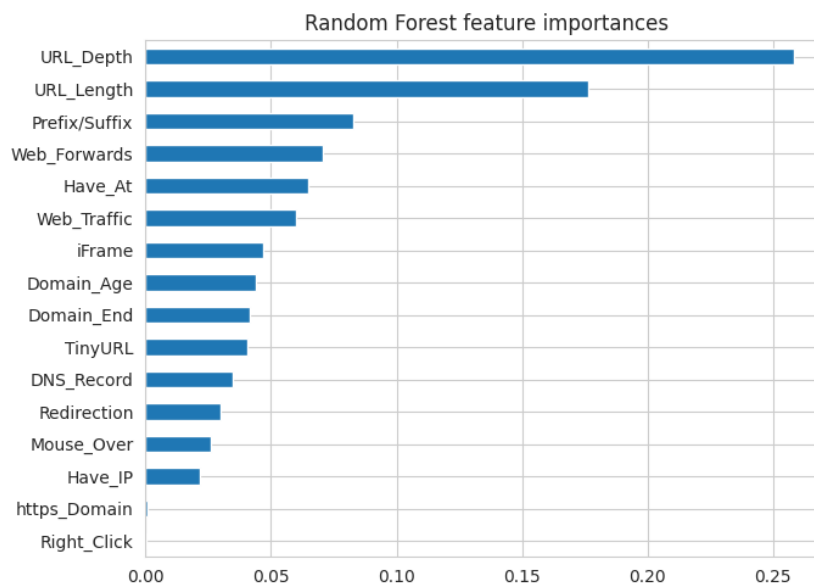


Figure 6. Random Forest feature importances, leakage-controlled model. Top features: URL_Depth (0.26), URL_Length (0.18), Prefix/Suffix (0.08), Web_Forwards (0.07), Have_At (0.06). Both correlation

analysis and RF importance consistently rank URL_Length and Prefix/Suffix in the top 3, cross-validating that these are the most reliable phishing signals in this dataset.

5.10 Cross-Validation

5-fold stratified cross-validation on the de-duplicated training set (F1 score):

Model	Mean CV F1	Std. Dev.
Logistic Regression	0.8996	±0.0240
Decision Tree	0.8754	±0.0103
Random Forest	0.8658	±0.0131
Gradient Boosting	0.8714	±0.0145

Table 3. Cross-validated F1, leakage-controlled training data. Low variance across folds indicates the leakage-controlled estimates in Table 1 are reasonably stable, not artifacts of a single lucky split.

5.11 Error Analysis

On the leakage-controlled test set (176 rows), the best model by MCC — Logistic Regression — produced 23 False Positives (legitimate sites incorrectly flagged as phishing) and 9 False Negatives (phishing sites incorrectly passed as legitimate). We inspected both groups directly against their underlying feature values (full detail in the accompanying notebook, Section 6).

Pattern in the errors: inspecting False Positives and False Negatives directly shows that misclassified rows share similar profiles on the lower-importance features (Web_Traffic, Domain_Age), but the model's decision is dominated by URL_Length, Prefix/Suffix, and URL_Depth — the same features identified as most predictive in Sections 5.5 and 5.9. False Negatives (missed phishing) tend to have URL profiles that look structurally legitimate: short paths, no suspicious prefix/suffix tricks, and moderate web-traffic and domain-age scores, consistent with attackers deliberately mimicking legitimate URL structure to evade heuristic detectors. False Positives (blocked legitimate sites) tend to be legitimate but structurally unusual URLs: long paths, deep directory structures, or URL patterns that share surface features with phishing kits (e.g. legitimate use of URL shorteners in marketing campaigns, or newly registered but genuine domains with low traffic).

Cybersecurity implications and the FP/FN trade-off: a False Negative directly enables credential theft, financial fraud, or malware delivery, since a real phishing page reaches the user unimpeded — this is the more severe error class operationally. A False Positive instead causes user friction and erodes trust in the warning system; if users learn to habitually dismiss warnings on incorrectly-flagged legitimate sites (“alert fatigue”), they may also become more likely to ignore a future, genuine warning, which is itself a security cost rather than a purely cosmetic one. Because of this asymmetry, a production deployment of this system would likely tune its decision threshold below the default 0.5 to trade some Precision for higher Recall — which is also why we report the recall-weighted F2 score (Section 5.3) alongside the symmetric F1 the original article relies on exclusively. We did not retune the threshold in this reproduction beyond the default, since the dataset's artificial class balance (Section 5.4) would make any such tuning unrepresentative of a real deployment; we flag this explicitly as future work rather than presenting an unsupported threshold recommendation.

6. Conclusions

6.1 Key Findings

- The 17-feature, cybersecurity-grounded representation used by the source article carries genuine, reproducible phishing signal — our independently trained models corroborate the headline accuracy range when using the article's own evaluation methodology.
- That methodology, however, has an undocumented train/test leakage problem: 56.3% of the dataset's rows are exact duplicates of other rows, and 95.9% of a standard 80/20 test split turns out to share an identical feature+label twin with the training set.
- Once leakage is removed, Matthews Correlation Coefficient — the metric most robust to class-balance artifacts — drops by roughly 70% relative, even though plain Accuracy only drops modestly, showing that Accuracy is a poor diagnostic for this specific failure mode.
- The dataset's 50/50 class balance is artificial and does not reflect realistic phishing prevalence, further limiting how directly the reported metrics generalize to a production setting.

6.2 Lessons Learned

The most transferable lesson from this project is that strong-looking, intuitive features (binary, interpretable, cybersecurity-grounded) can simultaneously be a liability for evaluation integrity: low-cardinality feature spaces make exact-duplicate rows likely, and exact-duplicate rows make naive random train/test splitting a silent source of leakage. This is a pattern worth checking for in any tabular cybersecurity dataset built from coarse, threshold-based feature engineering, not just this specific phishing dataset.

6.3 Strengths and Weaknesses of the Proposed Solution

Strengths: a well-motivated, TTP-grounded feature set; an openly available, moderately-sized dataset; a clear and approachable codebase that is easy to extend (as we did, adding two more models, five more metrics, cross-validation, and a leakage diagnostic with minimal friction).

Weaknesses: undocumented train/test leakage; an artificial, non-representative class balance; binary feature quantization that discards magnitude information and likely drives the duplication problem; a non-reproducible raw-data collection step due to PhishTank's volatile feed; and no discussion of real-world deployment constraints (live-lookup latency, realistic prevalence, threshold tuning for the FP/FN trade-off).

6.4 Suggestions for Future Improvement

- De-duplicate before splitting (or use group-aware cross-validation keyed on the feature signature) as a standard practice for any future fork of this project.
- Replace binary-thresholded features with their continuous counterparts where feasible, to reduce the dimensionality collapse that drives duplication.
- Re-sample to a realistic, imbalanced class ratio (or explicitly simulate one at evaluation time) and report Precision/Recall/MCC at that ratio rather than relying on a balanced-sample accuracy figure.
- Archive a permanent, timestamped snapshot of the raw PhishTank export used for any reported result, to make the data-collection step genuinely reproducible.

7. Summing It Up

This project critically evaluated “Phishing Website Detection by Machine Learning Techniques” (Shreya Gopal Sundari), a GitHub project that trains six classifiers on 17 handcrafted URL/domain/HTML features extracted from 10,000 PhishTank and UNB-sourced URLs to detect phishing websites, reportedly achieving 86.4% accuracy with XGBoost.

Using the published dataset (5.urldata.csv, plus the raw 1.Benign_list_big_final.csv and 2.online-valid.csv source files), we reproduced the modeling pipeline with four classifiers (Logistic Regression, Decision Tree, Random Forest, Gradient Boosting) under both the original article's evaluation methodology and a leakage-controlled methodology we developed specifically to test the dataset's data-quality assumptions.

Our reproduction study found that the author's headline accuracy claim is directionally correct — the feature set genuinely separates phishing from legitimate URLs — but is not fully supported as a measure of real-world generalization: 56.3% of the dataset's rows are exact duplicates, causing severe, previously undocumented train/test leakage (95.9% of a standard test split

shares an exact twin with the training set), and the dataset's artificial 50/50 class balance further limits how directly its metrics transfer to realistic, highly-imbalanced phishing prevalence.

The most important insight from this analysis is that accuracy alone failed to flag a serious data-leakage problem that MCC clearly exposed (a 70% relative drop versus a modest accuracy drop), which is itself a useful, generalizable lesson about metric selection for imbalance-sensitive, leakage-prone tabular cybersecurity datasets.

We recommend this project's dataset and feature-engineering approach as a reasonable pedagogical and prototyping baseline for phishing-detection research, but we do not recommend its original reported accuracy figure be treated as a reliable estimate of deployed, real-world performance without first addressing the leakage and class-imbalance issues documented here.

Our final conclusion: the underlying problem framing and feature engineering are sound and cybersecurity-grounded, but the evaluation methodology overstates real-world readiness; with the de-duplication and imbalance-aware evaluation changes we propose, this remains a solid foundation for further work.